Intro

이주현



화면이 ‘안 된다’로 끝나지 않게 만드는 개발자,
이주현입니다

주요 기술

TypeScript JavaScript React Next.js

- Server Route Handler에서 OAuth 인가 코드를
  세션 토큰으로 교환해 클라이언트 노출 차단

Zustand

- 스토어 라이프 사이클 통제
- URL uuid 변경 시 상태 초기화, 로드 상태 기반 스켈레톤 전환 연결

실시간 비동기 제어

- SSE 스트리밍 채팅(중단 / 실패 원인 3분류 / 재시도)
- Debounce + AbortController로 요청 경쟁상태 제어

React Native

- K-Route 모바일 앱 개발

협업 도구

Figma Notion Git

데이터

Supabase MySQL

학력

한성대학교

2022 - 2027 (컴퓨터공학부)

교육 사항

LG U+ 유레카 부트캠프 프론트엔드

2026. 04. 06 - 2026. 10. 28 (진행 중)

React/Next.js/TypeScript 기반 프론트엔드 교육 이수

팀 프로젝트 3건 수행

- Moonorang (2026. 08. 14 - 2026. 09. 04)
- NolMong (2026. 07. 16 - 2026. 07. 30)
- PORONG (2026. 05. 22 - 2026. 06. 04)

활동

2025 SW 인재페스티벌 (학교 대표 작품 선정)

2025. 09. 08 - 2025. 12. 03

ACK/ASK 2025 학술발표대회 (은상 수상 | 논문 게재)

2025. 09. 08 - 2025. 11. 08

2025. 03. 03 - 2025. 05. 31

활동

2025 SW 중심대학 디지털 경진대회 (학교 대표 작품 선정)

2025. 06. 02 - 2025. 08. 12

2025 관광데이터 활용 공모전

2025. 05. 05 - 2025. 11. 06

2025 한성대학교 캡스톤디자인

2025. 03. 03 - 2025. 05. 30

한성대학교 공학경진대회 (금상 수상)

2024. 06. 17 - 2024. 09. 26

전공 튜터링 (자료구조 튜터 활동)

2024. 03. 04 - 2024. 06. 28

컴퓨터 공학 스터디 동아리

2023. 09. 04 - 2024. 08. 31

Link

https://github.com/hana03030

https://hana030.tistory.com/

자격증

SQLD

2026. 06. 19 | 한국데이터산업진흥원

토익스피킹 IM 1

2026. 04. 25 | YBM 한국 토익위원회

정보처리기사

2025. 12. 24 | 한국산업인력공단

수상

ACK 2025 학술발표대회 은상

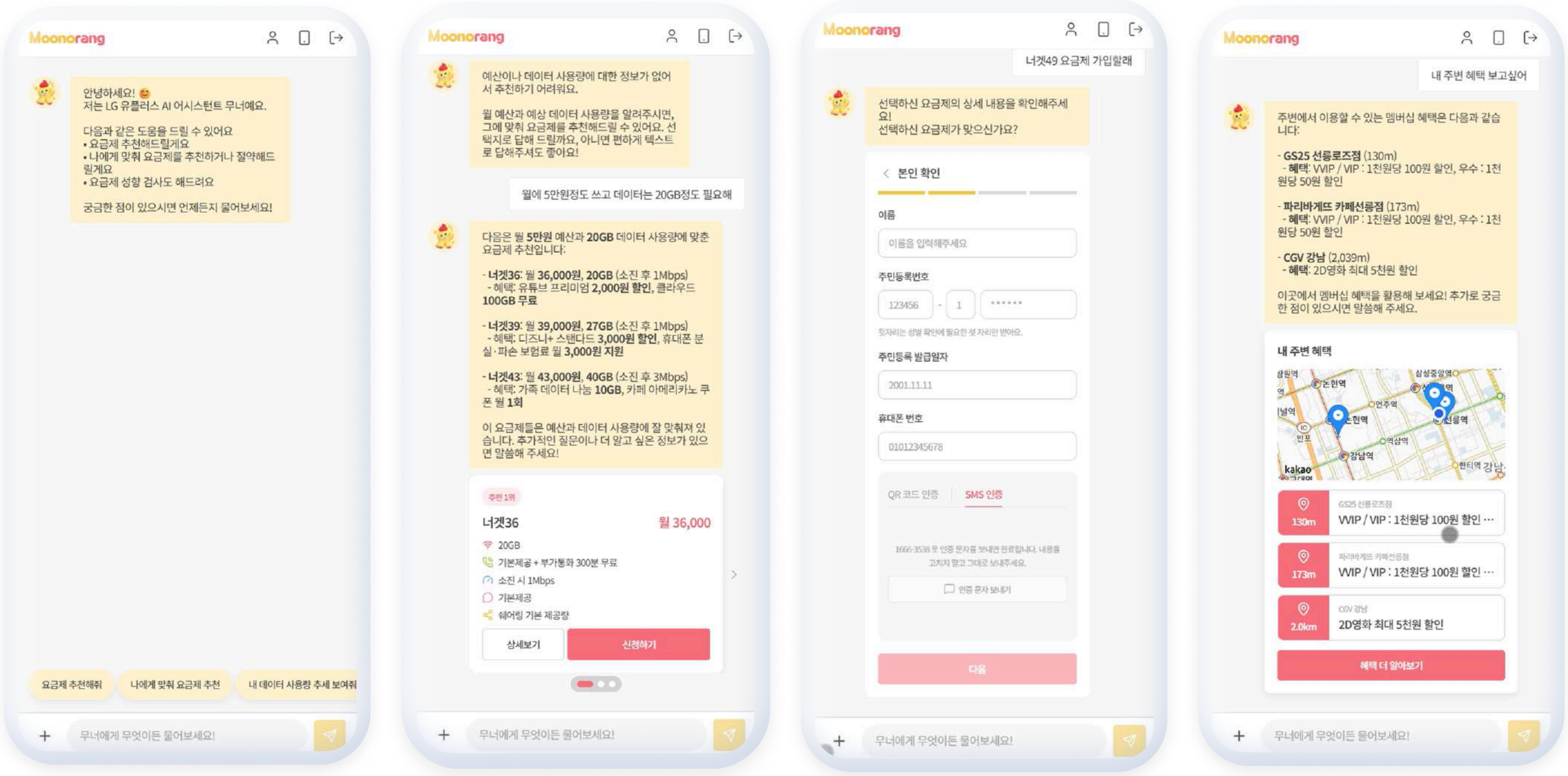
2025. 11. 08 | 한국정보처리학회 | 프로젝트 - SCOPE

한성대학교 공학경진대회 금상

2024. 09. 26 | 한성대학교 | 프로젝트 - 동화공작소

Moonorang

Project - 1



서비스

요금제 가입을 위한 AI 채팅 서비스

서비스 사이트 링크

작업 기간

2026. 08. 14 - 2026. 09. 04

문제 상황

여러 요금제의 월 요금, 데이터 제공량 등 사용자가 직접 비교해야하는 어려움.

요금제를 고른 뒤 별도의 페이지에서 가입 절차를 다시 진행해야 하는 번거로움.

AI를 도입하면서 요금제와 가격을 잘못 생성할 수 있어 신뢰도의 문제 발생.

목표

채팅 하나로 상담부터 가입까지 완료하는 웹 서비스 구현.

대화를 통해 예산과 데이터 사용량 등 조건 추출.

서버에서 계산을 통해 신뢰할 수 있는 추천.

응답을 글자 단위로 표시하여 기다림 없는 응답 생성.

조건, 요약, 최근 대화로 나눠 비용을 일정하게 유지하면서 AI가 대화를 보존하고 기억.

기술 스택

Next.js TypeScript TailwindCSS Zustand
Supabase Framer-Motion Notion Figma
Kakao Map

구성원

FullStack (팀원 3명)

관련 활동

LG U+ 유레카 부트캠프 프론트엔드

산출물

Github Figma 협업 노션 발표 자료 소개 영상

역할

Design

프로토타입 제작 (Figma)

UI/UX 디자인 (Figma)

FrontEnd

프론트엔드 개발 일정 관리 및 업무 배정

공용 컴포넌트 구현 (채팅 카드, 공통 입력창, 요금제 카드)

채팅 화면 구현

실시간 스트리밍 채팅 상태 관리

채팅 카드 구현 (요금제 추천, 선택형 조건 수집 카드)

대화 저장과 복구

예외 처리 및 UX 최적화

프로젝트 시연

BackEnd

백엔드 개발 일정 관리 및 업무 배정

OpenAI 연동 채팅 API 구현

조건 추출, 요금제 추천, 절약 분석 설계

요금제 추천 알고리즘 설계

대화 컨텍스트 관리

회원 대화 관리 (DB저장 및 복구, 대화 초기화)

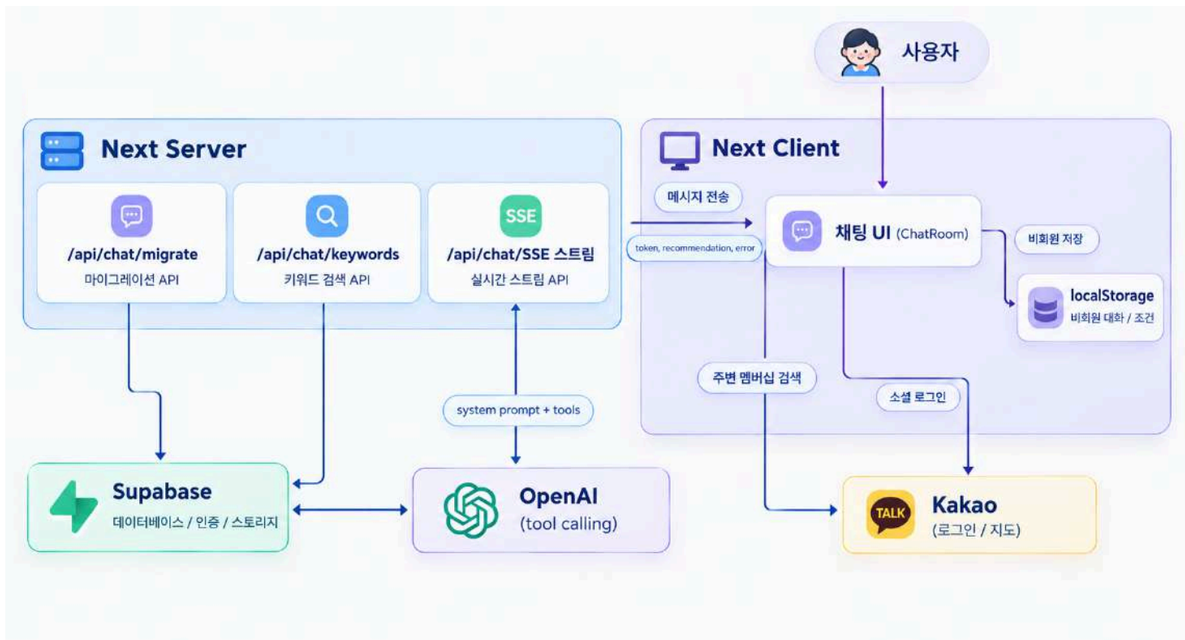
로그인 사용자 요금제 절약 상담 구현

카카오 로컬 API 연동

Supabase 데이터 조회 API

설계 문서와 배포 (DB 스키마 문서, 채팅 API 설계 문서 작성)

시스템 구조도



핵심 구현

실시간 스트리밍과 서버 검증 카드를 결합한 요금제 상담 채팅

사용하게 된 맥락

AI가 답변을 만드는 데 몇 초가 걸립니다. 답변이 다 완성될 때까지 빈 화면을 보여주면 사용자는 답답해서 떠나기 쉽습니다.
또 AI는 가끔 사실이 아닌 내용을 그럴듯하게 지어내기 때문에 요금제 이름이나 가격을 AI가 직접 쓰게 되면
잘못된 가격을 보고 가입하는 일이 생길 수 있습니다.
그래서 답변은 빠르게 보여주되, 요금 정보만큼은 AI가 아니라 실제 데이터에서 가져오는 구조를 만들고자 했습니다.

수행

- **답변을 글자 단위로 바로 표시**
  - 서버가 답변을 조금씩 나눠 보내는 방식인 SSE을 사용해, AI가 쓰는 글자를 받는 즉시 말풍선에 이어 붙였습니다.
  - 추천 요금제 카드도 준비되는 즉시 대화 사이에 끼워 넣었습니다.
- **멈추기, 실패 안내, 다시 시도**
  - 답변이 나오는 중에 사용자가 '멈추기'를 누를 수 있게 했습니다.
  - 실패하면 이유(응답 시간 초과, AI 서버 오류, 네트워크 연결 오류)를 구분해서 알려주고, 버튼 한 번으로 다시 시도할 수 있습니다.
- **AI는 설명만, 요금 정보는 실제 데이터로**
  - 요금제를 추천받을 때 몇 위로 추천할지는 서버가 데이터베이스의 실제 요금제 정보로 계산해 카드로 보여주도록 했습니다.
  - AI는 그 계산 결과를 받은 뒤에 "이 요금제는 데이터를 많이 쓰시는 분께 맞아요" 같은 설명 문장만 씁니다.
  - 혹시 AI가 이 규칙을 어기고 요금제 정보를 직접 쓰면, 그 답변은 사용자에게 보이기 전에 걸러냅니다.

결과

답변을 끝까지 기다릴 필요 없이 AI가 쓰는 즉시 글자가 나타나, 사용자가 대기 시간을 거의 느끼지 않는 상담 흐름을 완성했습니다.
응답이 늦어지면 최대 30초 안에 실패를 알리고 원인을 3가지로 구분해 안내해서, 어떤 오류 상황에서도 사용자가 멈추지 않고
버튼 한 번으로 상담을 이어갈 수 있게 했습니다.
무엇보다 카드의 요금 정보를 모두 실제 데이터에서 가져오도록 설계해, AI가 지어낸 가격이 사용자에게 보일 가능성을 원천 차단하고
서비스의 신뢰도를 확보했습니다.

[AI에게 추천 시점을 판단하는 조건만 제공]

```
export const RECOMMEND_PLANS_TOOL: ChatCompletionTool = {
  type: 'function',
  function: {
    name: 'recommend_plans',
    description:
      '사용자가 이번 대화에서 직접 말해준 조건(예산, 예상 데이터/테더링 사용량)을 기준으로
      ' +
      '요금제 추천을 원할 때 호출한다. 실제 추천 요금제·순위는 서버가 계산한다. ' +
      '조건을 아직 잘 모르겠으면 호출하지 말고 먼저 물어본다. "내 실제 이용 데이터/사용
      패턴에 ' +
      '맞춰서" 추천해달라는 뜻이면(로그인 사용자의 실제 사용 이력 기반) 이 tool이 아니라 '
      +
      'analyze_savings를 호출한다 - 둘은 근거 데이터가 다르다.',
    parameters: {
      type: 'object',
      properties: {
        scope: {
          type: 'string',
          enum: ['recommended', 'catalog', 'alternative'],
          description:
            '평소의 "지금까지 파악된 조건으로 추천" 요청이면 이 필드 자체를 넣지 않는다. '
            +
            '아래 세 경우에만 채운다. "recommended": "그중에 제일 비싼/싼 거", "추천해준
            ' +
            '것 중에서"처럼 방금 추천한 요금제들 범위 안에서만 다시 고를 때. "catalog": '
```

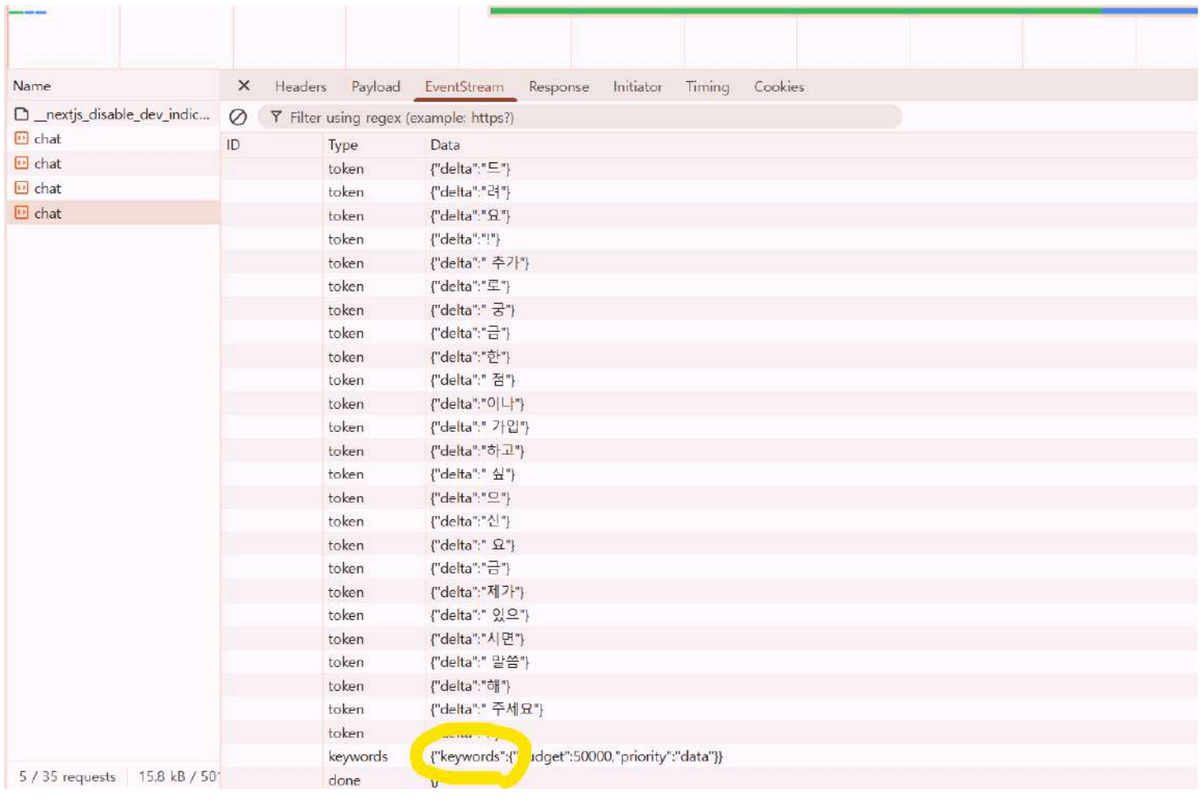
[서버가 DB 데이터로 추천을 계산해 카드로 먼저 보내고, 결과를 AI에게 전달]

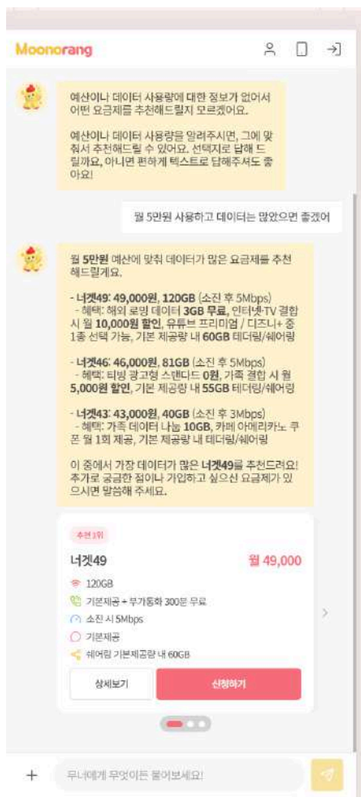
```
const {
  recommendations: scored,
  didRelaxBudget,
  didRelaxDataUsage,
  didRelaxTethering,
  isInterestBrowse,
  effectivePriority,
} = selectRecommendedPlans(plans, keywords);

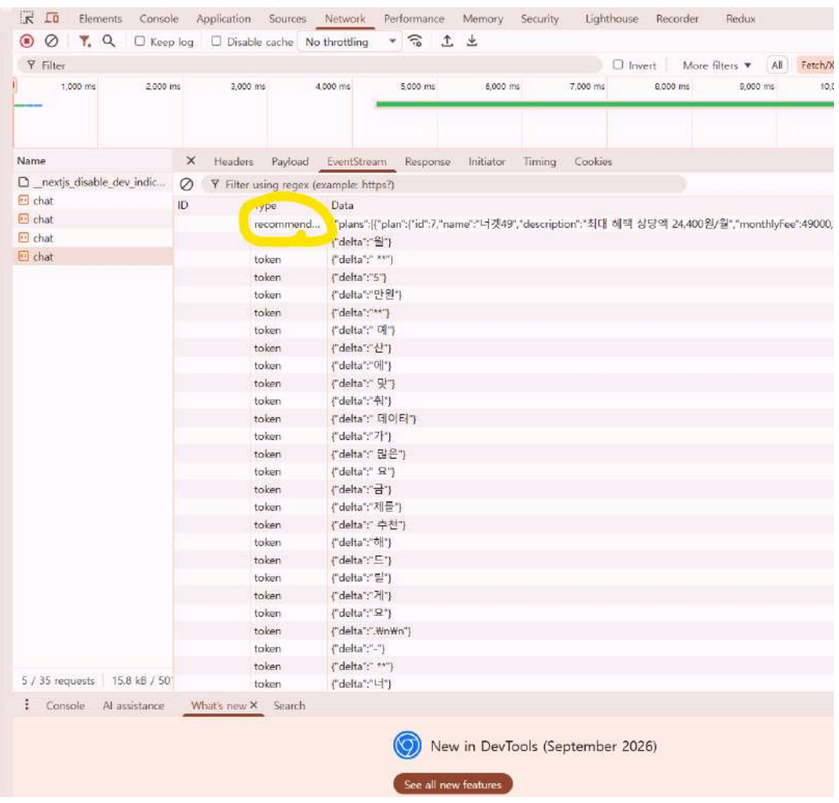
const recommendations: PlanRecommendation[] = scored.map((item) => ({
  plan: item.plan,
  rank: item.rank,
  reason: isInterestBrowse
    ? describeInterestMatch()
    : effectivePriority === 'data'
      ? describeDataPriorityMatch(item.isWithinBudget)
      : effectivePriority === 'priciest'
        ? describePriciestMatch()
        : effectivePriority === 'cheapest'
          ? describeCheapestMatch()
          : describeFit(item.isWithinBudget, usageGb),
}));

send({ event: 'recommendation', data: { plans: recommendations } });
```

[keywords 수집 → recommendation 카드 순서로 이벤트 작동]







[최종 화면과 DB 값 비교 - 카드의 제공량이 DB값과 똑같음 (AI가 지어내는 가능성 차단)]



name
varchar
너겟49

description
varchar
최대 혜택 상당액 24,400원/월

monthly_fee
int4
49000

data_allowance
varchar
120GB (소진 후 5Mbps)

voice_sms
varchar
기본제공 / 기본제공 / 300분 무료


```
{
  "roaming": "해외 로밍 데이터 3GB 무료",
  "home_bundle": "인터넷·TV 결합 시 월 10,000원 할인",
  "media_contents": "유튜브 프리미엄 / 디즈니+ 중 1종 선택",
  "max_benefit_value": "24,400원/월",
  "tethering_sharing": "기본제공량 내 60GB"
}
```

대화가 길어져도 앞 내용을 잊지 않는 챗봇

사용하게 된 맥락

AI는 이전 대화를 스스로 기억하지 못합니다. 그래서 질문할 때마다 지난 대화를 함께 보내줘야 합니다.
하지만 대화가 길어질수록 보내는 양이 늘어나 비용이 커지고 답변 품질도 떨어졌습니다.
반대로 보내는 양을 줄이면, 사용자가 앞에서 말한 예산이나 데이터 사용량을 잊어버려 같은 질문을 반복하게 됐습니다.
또 비회원의 대화는 사용자 기기(브라우저)에 저장하는데, 대화가 끝없이 쌓이면 저장 공간이 부족해질 수 있었습니다.

수행

- **기억을 세 가지로 나눠 관리**
  - 꼭 기억할 조건: 예산, 데이터 사용량처럼 상담 내내 필요한 정보는 따로 저장하고, 사용자가 바꾸면 최신 값으로 교체했습니다.
  - 지난 대화 요약: 오래된 대화는 몇 문장짜리 요약으로 압축했습니다.
  - 최근 대화 원문: 가장 최근 대화는 요약하지 않고 그대로 보냈습니다.
- **AI의 기억과 화면에 보이는 대화를 분리**
  - 화면과 브라우저 저장에는 최근 24번의 대화만 남기고, 그보다 오래된 대화는 정리했습니다.
  - 이때 이미 요약에 반영된 대화만 지웠습니다. 화면에서는 사라져도 AI는 요약을 통해 내용을 계속 기억합니다.
- **과거 대화를 읽는 중에는 정리하지 않기**
  - 사용자가 위로 스크롤해서 예전 대화를 읽고 있을 때 오래된 대화를 지우면, 읽던 내용이 갑자기 사라지거나 화면이 튀는 경험을 하게 됩니다.
  - 그래서 대화 정리는 스크롤이 맨 아래에 있을 때만 실행하고, 위를 보고 있는 동안에는 미뤘다가 다시 맨 아래로 돌아왔을 때 정리하도록 했습니다.
- **로그인하면 대화를 이어서**
  - 회원의 대화는 데이터베이스에 모두 저장해서, 화면에서 정리되더라도 원본은 사라지지 않게 했습니다.
  - 로그인 전에 나눈 대화는 로그인하는 순간 회원 계정으로 옮겨, 처음부터 다시 설명할 필요 없이 상담을 이어가게 했습니다.

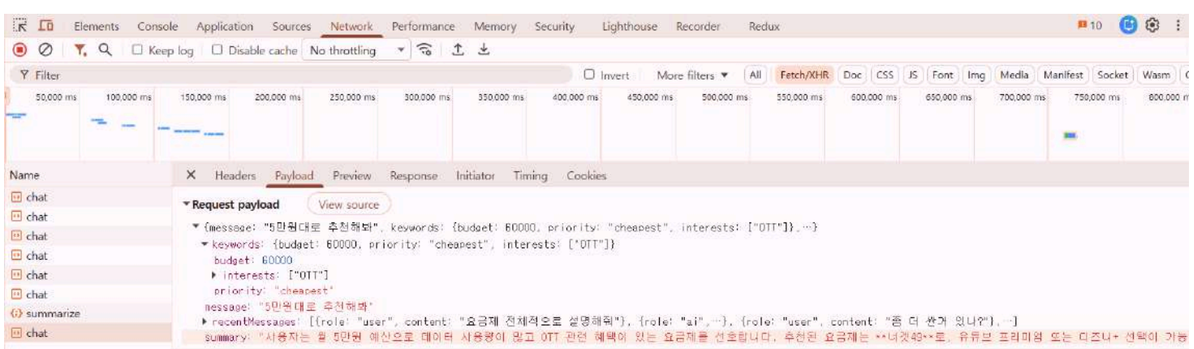
결과

대화가 길어져도 AI에 원문으로 보내는 대화를 최대 7턴으로 고정해, 30턴 대화 기준 전송량을 약 77% 줄이면서도 사용자가 처음 말한 조건까지 끝까지 기억하는 챗봇을 만들었습니다.

화면에는 최근 24턴만 남겨 대화가 길어져도 화면이 가볍게 유지되도록 했고, 정리는 스크롤이 맨 아래일 때만 실행해 읽던 대화가 갑자기 사라지는 일 없이 자연스러운 사용 경험을 지켰습니다.

또 로그인 전후와 페이지 이동 후에도 대화가 그대로 이어져, 사용자가 같은 설명을 반복할 필요가 없어졌습니다.

[AI에 보내는 요청 - 조건, 요약, 최근 대화가 분리되어 전달됨]



[스크롤이 맨 아래일 때만 이미 요약된 대화만 화면에서 정리]

```
// 화면 유지 상한을 넘긴 오래된(이미 요약된) 턴은, 사용자가 맨 아래를 보고 있을
때만
// 걷어낸다 - 과거 대화를 스크롤해서 보는 도중에 눈앞에서 사라지는 걸 막기 위함.
useEffect(() => {
  if (isAtBottom) pruneVisibleMessages();
}, [isAtBottom, messages.length, pruneVisibleMessages]);

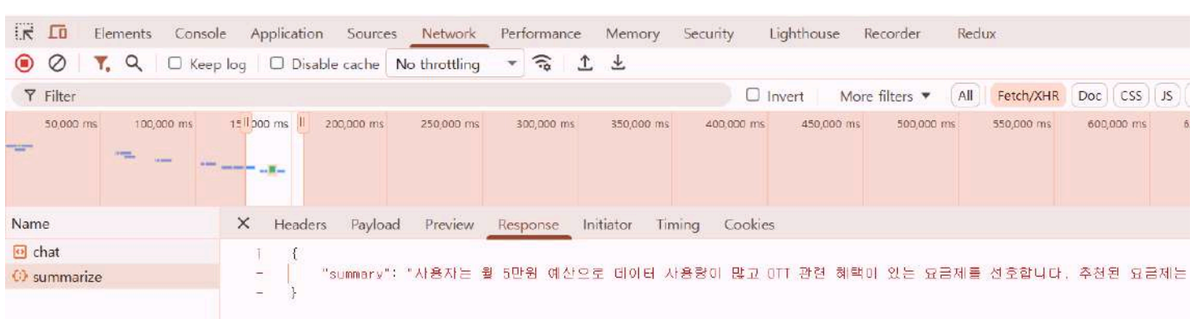
const lastMessage = messages[messages.length - 1];
const lastMessageId = lastMessage?.id;
```

```
export function pruneSummarizedMessages(
  messages: ChatMessage[],
  summarizedTurnCount: number,
  maxVisibleTurns: number,
): PruneResult {
  const overflowTurns = countTurns(messages) - maxVisibleTurns;
  const removedTurns = Math.max(0, Math.min(overflowTurns,
  summarizedTurnCount));

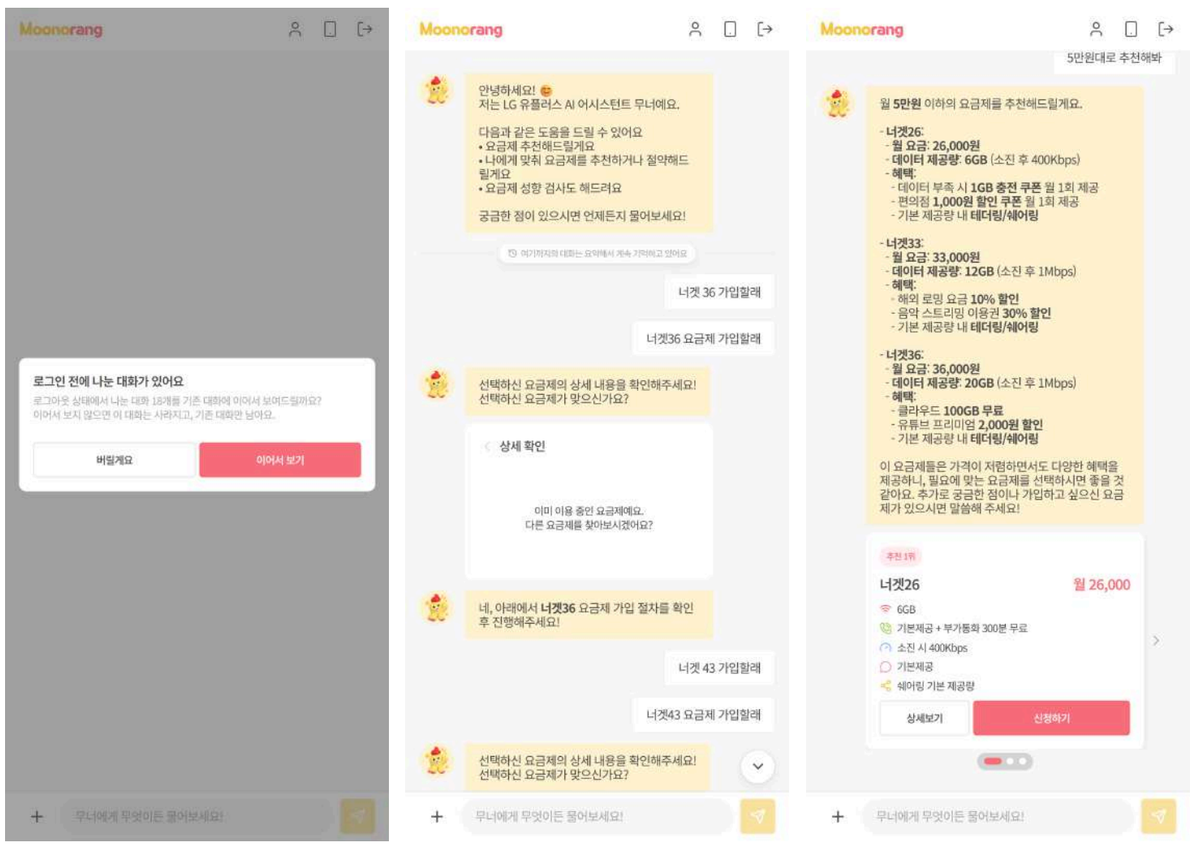
  if (removedTurns === 0) return { messages, removedTurns: 0 };

  return { messages: messages.slice(removedTurns * 2), removedTurns };
}
```

[답변이 끝난 뒤 요약이 별도로 실행되어 응답 속도에 영향 없음]



[로그인 전 나눈 대화를 로그인 후 버릴지, 이어서 볼지를 선택할 수 있으며
앞서 말한 조건을 기억함]



트러블 슈팅

챗봇이 방금 한 대화를 잊어버리는 문제

상황

대화 요약 기능을 넣은 뒤, 챗봇이 방금 나눈 대화나 처음에 말한 조건을 기억하지 못하는 문제가 생겼습니다.
예를 들어 "아까 추천해준 것 중에 두 번째 거"라고 물으면 엉뚱한 답을 했고, 대화가 길어지면 처음 말한 내용을 잊어버렸습니다.

원인 파악

AI에게 실제로 무엇을 보내고 있는지 확인해 보니 두 가지 문제가 있었습니다.
첫째, 아무도 챙기지 않는 구간이 있었습니다. 요약은 대화가 8번 쌓여야 만들어지는데, 그 전까지의 최근 대화는 요약에도 없고 원문으로도 보내지 않고 있었습니다. AI 입장에서는 그 대화가 아예 없었던 셈입니다.
둘째, 요약이 이전 요약을 지워버렸습니다. 두 번째 요약부터 AI가 기존 요약에 새 내용을 더하지 않고, 새 대화만으로 요약을 새로 써버렸습니다.
그래서 초반 대화 내용이 사라졌습니다.

해결 방안

- **아직 요약되지 않은 대화를 원문 그대로 함께 전송**
  - 마지막 요약 이후의 대화를 골라내는 기능을 만들어, 질문할 때마다 함께 보냈습니다.
  - 전송에 실패해 비어 버린 말풍선처럼 의미 없는 내용은 걸러냈습니다.
- **요약을 덮어쓰지 않고 이어 쓰도록 지시**
  - AI에게 "기존 요약 내용은 유지하고, 새 대화 내용을 합쳐서 처음부터 지금까지의 요약을 써라"라고 명확히 지시했습니다.

결과

AI가 볼 수 없던 최대 7턴의 기억 공백을 완전히 없애, 모든 대화가 요약이나 원문 중 한 곳에 반드시 전달되도록 했습니다.
그 결과 "아까 추천해준 거"처럼 앞 대화를 가리키는 질문에도 자연스럽게 답하는, 실제 상담원처럼 맥락을 이어가는 대화가 가능해졌습니다.

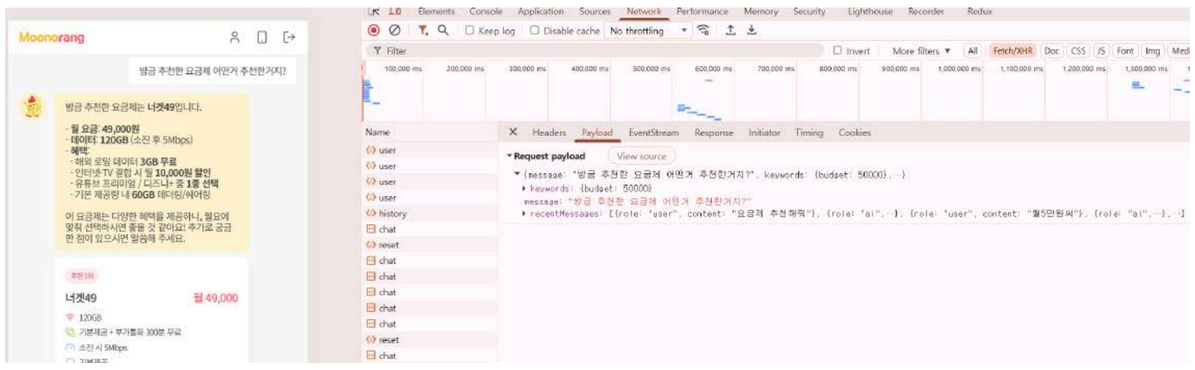
[12번째 질문 시점에 AI가 받는 내용
- 수정 전에는 아직 요약되지 않은 6~11턴이 빠져 공백이 생김]



[수정 전: 조건과 현재 질문만 전달되고 이전 대화(recentMessages)는 빠져 있음
그렇기에 방금 추천한 요금제를 가리키는 질문에 맥락을 잃고 엉뚱하게 답변]



[수정 후: 아직 요약되지 않은 이전 대화를 recentMessages로 함께 전달
같은 질문에 앞서 추천한 요금제를 기억하고 설명]



트러블 슈팅

모든 질문마다 위치 권한을 물어봐서 답변이 늦어지던 문제

상황

"내 주변 멤버십 혜택 찾기" 기능을 추가한 뒤, 위치와 상관없는 "요금제 추천해줘" 같은 질문에도 "위치 정보를 허용하시겠습니까?" 팝업이 떴습니다.
사용자가 팝업에 답하기 전까지는 질문이 전송되지 않아 답변이 늦어졌습니다.
또 위치를 "허용"했는데도 위치를 못 가져오는 경우가 있었지만, 아무 안내 없이 실패해서 이유를 알 수 없었습니다.

원인 파악

위치가 필요한 질문인지는 AI가 질문 내용을 이해한 뒤에야 알 수 있습니다.
그런데 화면 쪽에서는 이를 알 수 없어서, 모든 질문을 보내기 전에 일단 위치부터 물어보고 기다리고 있었습니다.
또 브라우저에서 위치를 허용해도, 컴퓨터 자체의 위치 기능이 꺼져 있으면 위치를 가져올 수 없다는 점도 확인했습니다.

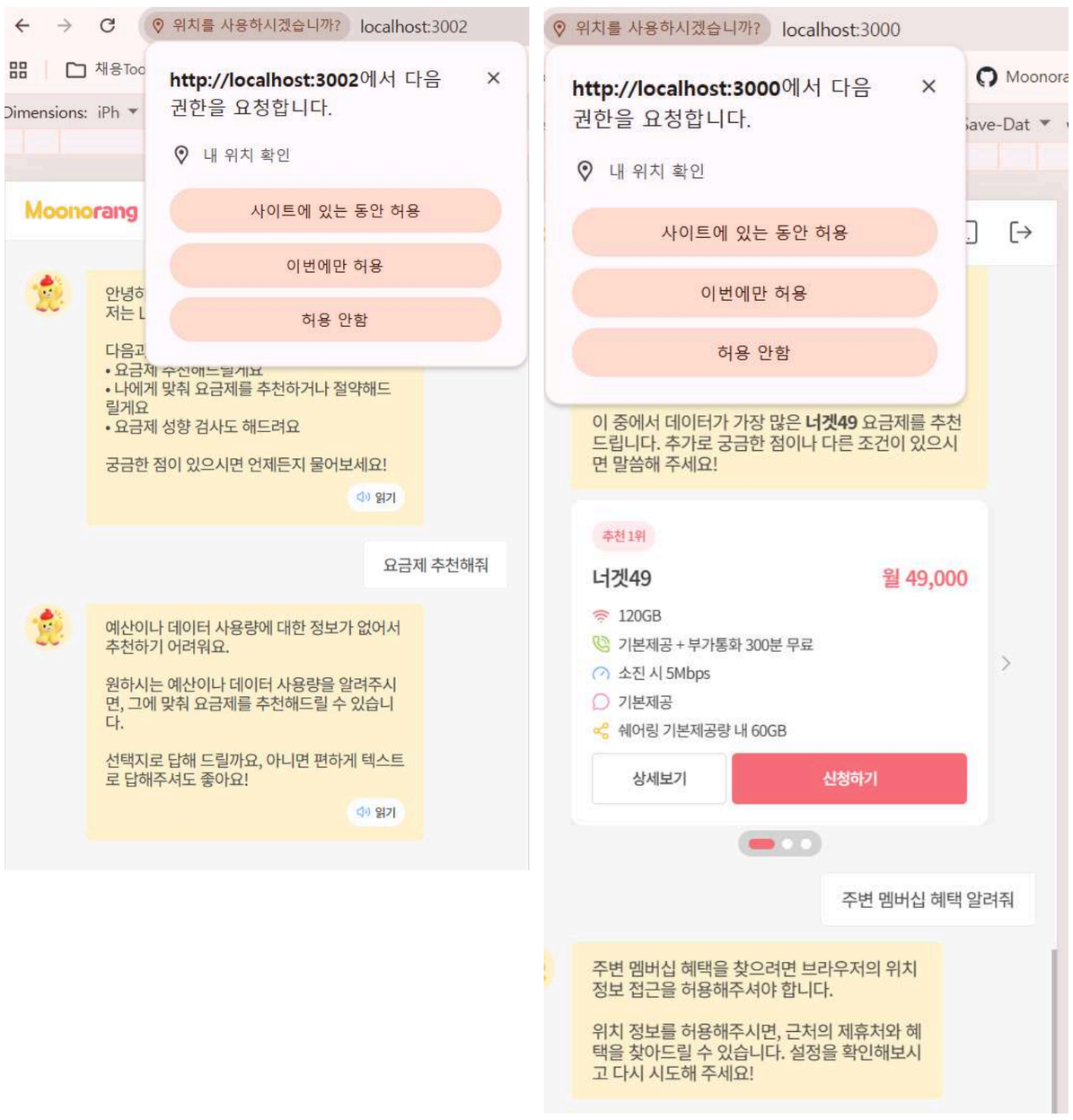
해결 방안

- **위치가 정말 필요할 때만 알려주기**
  - AI가 "주변 혜택 찾기" 요청이라고 판단했는데 위치 정보가 없을 때만, 서버가 화면에 "위치가 필요해요"라는 신호를 보내도록 했습니다.
- **신호를 받았을 때만 위치 권한 요청**
  - 모든 질문 전에 위치를 묻던 동작을 없애고, 서버의 신호를 받았을 때만 권한을 요청하도록 바꿨습니다.
  - 한 번 얻은 위치는 다시 묻지 않고, 팝업이 떠 있는 동안 또 요청이 와도 팝업이 두 번 뜨지 않게 했습니다.
- **실패 이유를 구분하고 다시 시도할 수 있게**
  - 실패 이유를 "권한 거부", "컴퓨터 위치 기능 꺼짐", "시간 초과"로 나눠 기록했습니다.
  - 실패해도 다음에 주변 혜택을 다시 물어보면 자연스럽게 다시 시도되도록 했습니다.

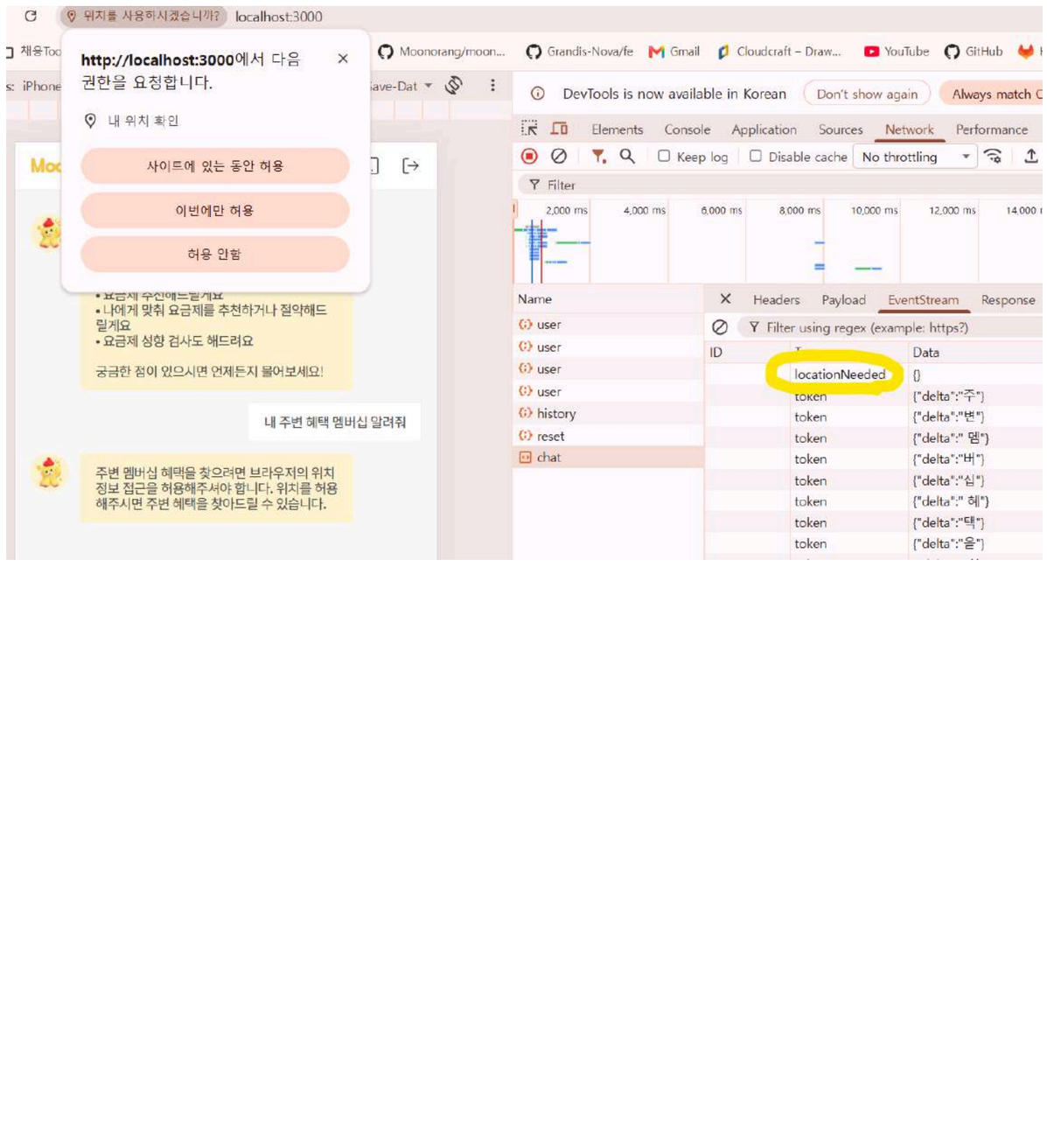
결과

위치와 상관없는 질문에서는 권한 팝업이 한 번도 뜨지 않도록 바꿔, 대부분의 상담이 팝업에 막히지 않고 곧바로 답변을 받게 됐습니다.
주변 혜택을 물어본 순간에만, 그것도 단 한 번만 권한을 요청해 사용자가 왜 위치가 필요한지 이해한 상태에서 허용할 수 있는 흐름을 만들었습니다.
위치를 가져오지 못한 경우에도 원인을 3가지로 구분해, "허용했는데 왜 안 되지?" 하는 상황을 바로 파악할 수 있게 했습니다.

[수정 전: 위치와 무관한 “요금제 추천해줘”에도 위치 권한 팝업이 뜨고 답변이 멈춤
수정 후: 요금제 추천은 팝업 없이 답변하고, 주변 혜택을 물어볼 때만 권한 요청]



[서버가 위치가 실제로 필요할 때만 보내는 locationNeeded 신호]



회고

AI에게 맡길 일과 코드로 보장할 일의 경계를 먼저 정했어야 했습니다

프로젝트 초반에는 "예산만 말하면 비싼 순으로", "1개만 달라면 1개만 추천", "요금제 정보는 반드시 도구를 거쳐서" 같은 상담 규칙을 대부분 AI에게 주는 지시문(시스템 프롬프트)에 적어 해결하려 했습니다. 그 결과, 한 가지 규칙을 고치면 다른 상황에서 오류가 새로 생기는 일이 반복됐습니다.
불가능한 요청에 대한 안내 문구를 강화하자 요금제 정보를 걸러내는 가드레일에 걸리는 응답이 늘어났고, 예산만 말한 요청에 우선순위를 설정하라는 지시는 5번 중 한 번도 지켜지지 않았습니다. 수정할 때마다 같은 질문을 손으로 다시 보내 확인하다 보니 QA에 드는 시간도 계속 늘어났습니다.
결국 순위 계산, 추천 개수, 우선순위 판단처럼 반드시 지켜져야 하는 규칙은 서버 코드로 옮기고, AI에게는 의도 파악과 설명만 맡기는 구조로 바꾸고 나서야 결과가 안정됐습니다.
AI를 활용하는 서비스일수록, 기능을 구현하기 전에 "무엇을 AI의 판단에 맡기고 무엇을 코드로 보장할지"를 먼저 나누는 설계가 서비스 전체의 신뢰도를 좌우한다는 것을 깨달았습니다.

지난 회고를 바탕으로 이번 프로젝트에서 이렇게 바꿨습니다

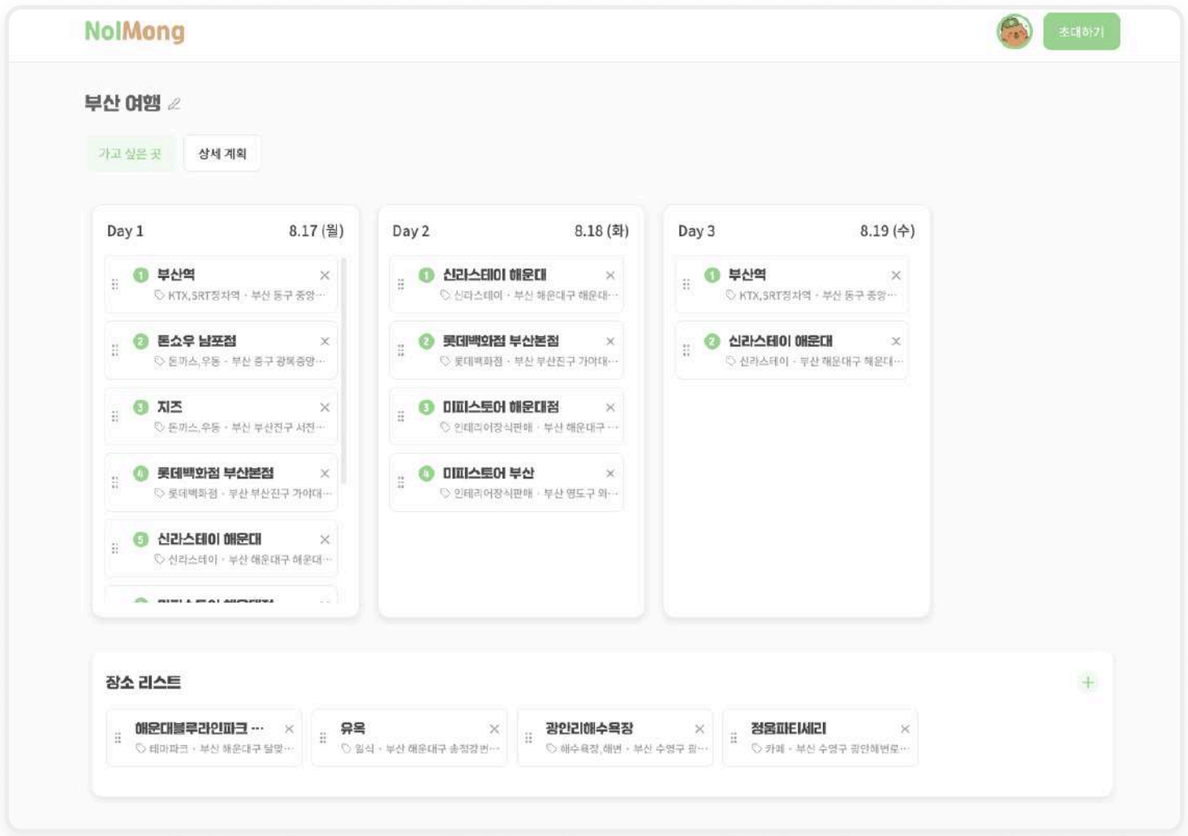
NolMong에서는 상태의 소유권과 동기화 규칙을 정하지 않고 구현을 시작한 탓에 잔상 노출과 데이터 덮어쓰기 버그를 겪었습니다.
그래서 Moonorang에서는 코드를 쓰기 전에 채팅 API 설계 문서부터 작성했습니다.
회원 대화는 DB, 비회원 대화는 브라우저 저장소를 유일한 원본으로 정했습니다.
로그인 여부가 확인되기 전에는 대화를 복구하지 않게 해서 화면 깜빡임을 막았습니다.
로그인 시 게스트 대화와 회원 대화가 충돌하면 자동으로 합치지 않고 사용자에게 선택하게 했습니다.
응답 실패 원인도 응답 시간 초과, AI 서버 오류, 네트워크 연결 오류로 먼저 정의한 뒤 화면 처리를 구현했습니다.

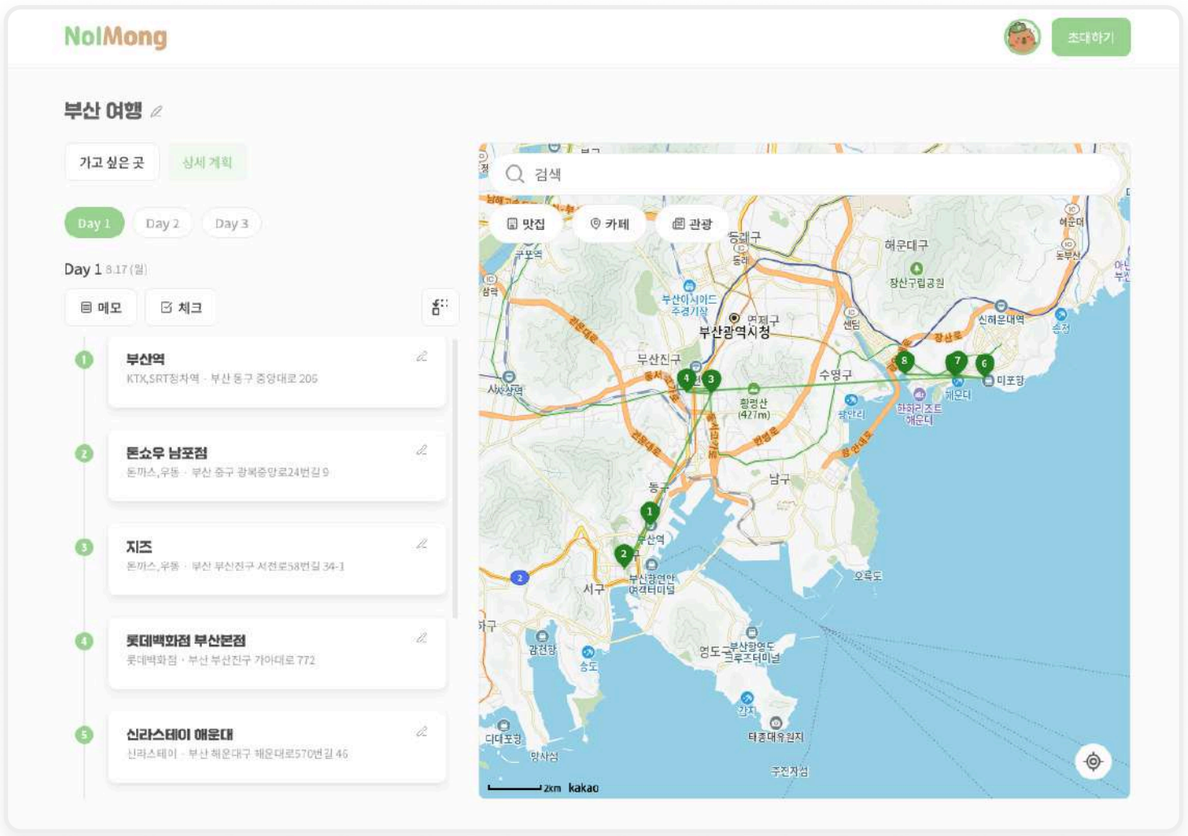
다음 프로젝트에서는 이렇게 바꿔볼 생각입니다

다음 프로젝트에서는 AI 기능을 설계할 때, 기능마다 AI가 판단할 부분과 코드가 책임질 부분의 경계를 개발 전에 문서화하는 단계를 필수로 넣으려 합니다. 또한 요구사항의 검증 기준을 처음부터 자동으로 실행되는 시나리오 테스트로 만들어, 프롬프트를 고칠 때마다 다른 상황이 깨지지 않았는지 바로 확인할 수 있게 할 생각입니다.
마지막으로 첫 응답 시간, 입력 토큰 수 같은 지표를 개발 초기부터 기록해, 구조를 바꿀 때마다 성능과 비용이 실제로 얼마나 개선됐는지 수치로 비교하며 개선해 나갈 것입니다.

NolMong

Project - 2





서비스

실시간으로 일행과 함께 여행 계획을 기획하는 플랫폼

서비스 사이트 링크

작업 기간

2026. 07. 16 - 2026. 07. 30

문제 상황

기존에는 계획을 짤 때 메신저와 지도 앱을 번갈아가며 계획해야하여 비효율적.

한 명만 모든 일정 계획을 떠맡게 되는 총대 구조.

복잡한 텍스트 중심 일정표로 인한 직관성 부족.

목표

Supabase 및 Ably를 활용한 프론트엔드 단 중심의 Next 컴포넌트 설계 및 상태 관리 최적화.

일행 간 실시간 편집 및 Kakao Maps 루트 동선 시각화 제공.

기술 스택

Next.js TypeScript Ably TailwindCSS

Zustand Supabase Figma Notion

Kakao Map Kakao OAuth Framer-Motion

구성원

FullStack (팀원 3명)

기여도 33%

관련 활동

LG U+ 유레카 부트캠프 프론트엔드

산출물

Github Figma 협업 노션 발표 자료 소개 영상

담당 역할

Design

프로토타입 제작 (Figma)

UI/UX 디자인 (Figma)

FrontEnd

프론트엔드 개발 일정 관리 및 업무 배정

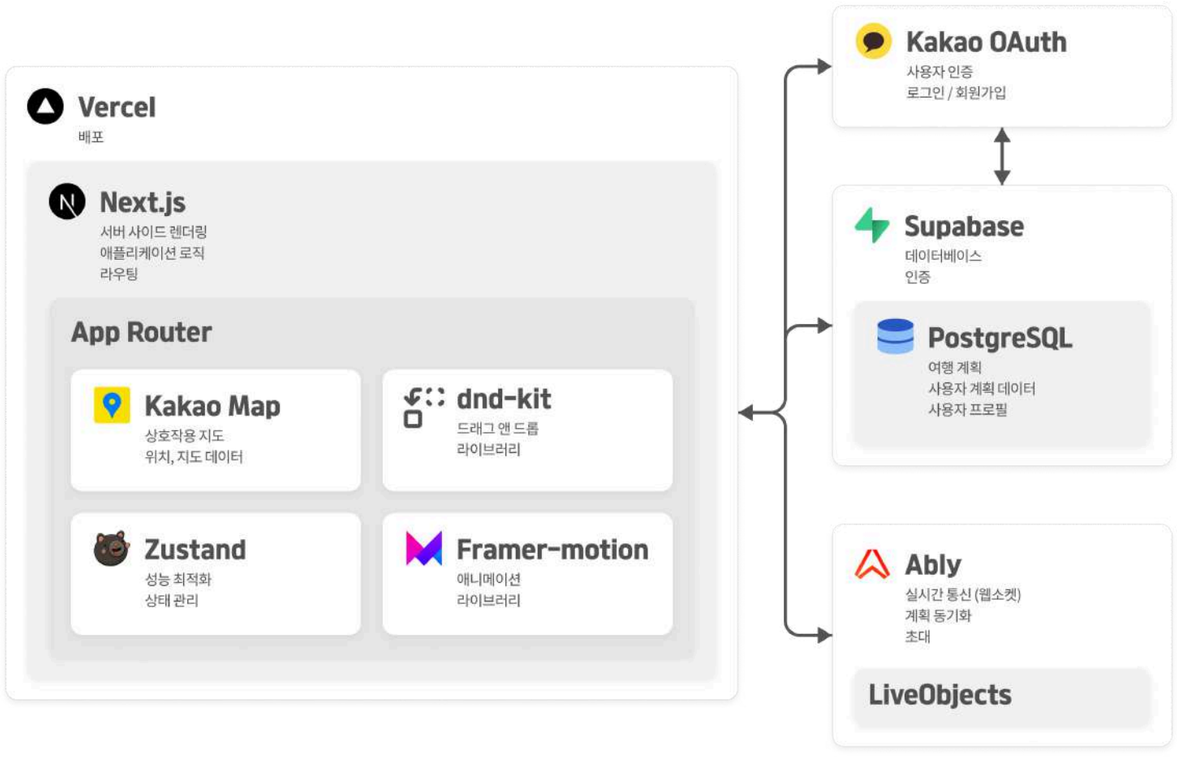
공용 컴포넌트 구현 (일자 카드, 장소 카드, 계획 카드)

초대하기 기능(링크, 수락 모달) 구현

계획 나가기 확인 모달 및 삭제 후 UI 상태 동기화 구현

실시간 드래그 앤 드롭 기능 구현

시스템 구조도



BackEnd

Supabase 연동, 계획 카드 Supabase sql 구현

Kakao OAuth 연동 (회원가입/로그인/로그아웃) 구현

Kakao OAuth 인증 핸들러 개발

Supabase 기반 초대 수락 및 멤버 참여 비즈니스 로직 개발

Supabase 기반 계획 나가기 비즈니스 로직 개발

핵심 구현

사용자 경험을 개선한 외부 초대 링크 및 즉시 유입 시스템 구축

사용하게 된 맥락

외부 초대 링크를 받은 사용자가 플랫폼에 들어올 때,
로그인이나 회원가입 절차를 거치면서 원래 초대받았던 여행 계획 초대를 잃어버리는 일이 발생했습니다.
이는 로그인을 한 뒤 초대 링크를 다시 눌러야 하는 불편함이 있다고 생각하여,
로그인이 되어도 초대 받은 여행 계획으로 곧바로 입장할 수 있는 유저 진입 경험을 제공하고자 했습니다.

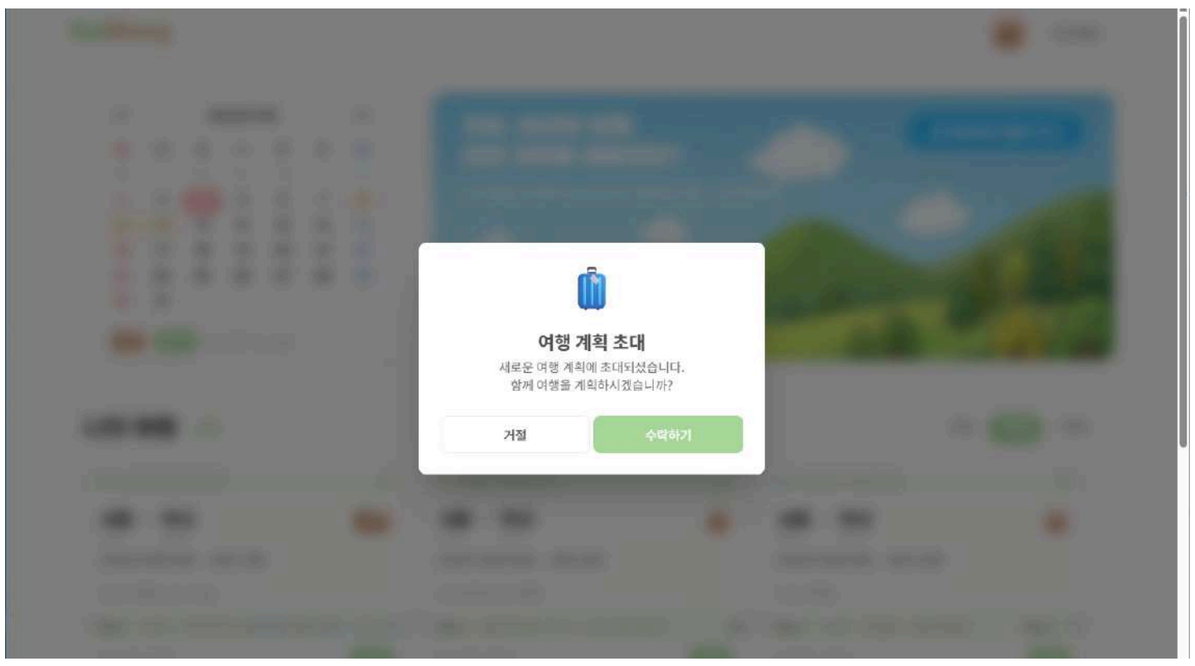
수행

- **초대 정보 보존 리다이렉트 설계**
  - 외부 공유 링크로 진입 시, 로그인 여부 및 신규 회원 여부를 파악했습니다.
  - 비 로그인/신규 유저의 경우 OAuth 로그인 페이지로 이동시키되, 쿼리스트링과 쿠키를 활용해 초대 URL을 보관했습니다.
- **Next.js Server Route Handler 및 인증 통합**
  - Kakao OAuth 인가 코드를 서버 단에서 안전하게 세션 토큰으로 교환 후, 유저 상태에 맞춰 원래 가려고했던 초대 링크로 리다이렉트 시켰습니다.
- **즉시 유입 모달 연동**
  - 최종 랜딩 완료 시 초대 ID를 감지하여, 사용자가 바로 참여 여부를 결정할 수 있는 초대 수락 모달을 화면상에 즉시 띄우도록 처리했습니다.

결과

로그인과 회원가입 단계를 거치더라도 초대받은 맥락이 유지되어,
로그인 직후 초대 모달을 통해 즉시 초대 계획으로 들어갈 수 있는 직관적인 유저 진입 경로를 완성했습니다.
이 과정에서 OAuth 인증 인가 코드를 서버 단에서 처리하여 클라이언트 노출 위험까지 함께 차단했습니다.

[초대 링크를 통해 로그인을 성공한 뒤 나오는 초대 모달]



[초대 링크 쿼리 파라미터를 파싱하는 로직]

```ts
export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const code = searchParams.get('code');
  const invite = searchParams.get('invite');

  // base url 구하기 (로컬 버셀)
  const baseUrl = getBaseUrl();

  const rawNext =
    searchParams.get('next') || searchParams.get('redirectTo') || '/main';

  const validNextPath =
    rawNext.startsWith('/') && !rawNext.startsWith('//') ? rawNext : '/main';
  const targetMainUrl = new URL(validNextPath, baseUrl);
  // 초대 UUID 추출
  const inviteUuid = targetMainUrl.searchParams.get('invite');
```

핵심 구현

Zustand 라이프사이클 관리 및 Skeleton UI를 활용한 데이터 잔상 제거와 UX 최적화

사용하게 된 맥락

데이터가 Zustand에서 관리되다 보니, A 여행 계획 페이지를 보다가 B 여행 계획 페이지로 이동할 때
이전에 조회했던 A 계획 데이터가 화면에 잠깐 떠 있다가 B 데이터로 바뀌는 잔상 현상이 발생했습니다.
이로 인해 사용자가 잘못된 정보를 순간적으로 보게 되는 혼란을 방지하고자 했습니다.

수행

- **스토어 상태 초기화 도입**
  - 여행 계획 페이지 진입 시 URL의 uuid가 변경될 때마다 useEffect에서 상태 초기화를 호출하도록 구현했습니다.
    이를 통해 이전 데이터를 즉시 비우고 로드 상태를 관리하여 잔상을 완전히 제거했습니다.
- **Custom Skeleton UI & framer-motion 전환**
  - 렌더링 레이아웃에 맞춰 가고 싶은 곳과 상세 계획 스켈레톤 UI를 제작했습니다.
    Zustand에서 관리하는 로드 상태에 맞춰 애니메이션을 넣어 스켈레톤과 실제 데이터 간의 페이드인 전환을 연결했습니다.

결과

잔상이 발생할 수 있는 구간 자체를 제거했습니다.
계획 페이지 진입 시 URL의 uuid 변경을 감지해 resetPlan으로 스토어를 먼저 비우고, 로드 상태가 끝나기 전까지는 스켈레톤만 렌더합니다.
이전 계획 데이터가 화면에 올라갈 수 있는 타이밍이 구조적으로 존재하지 않게 했습니다.
스켈레톤은 실제 렌더링 레이아웃(가고 싶은 곳 / 상세 계획)에 맞춰 제작해, 데이터가 도착했을 때 레이아웃이 밀리지 않고 페이드인으로 이어집니다.

[plan id(uuid)가 변경 시 resetPlan을 실행하여
스켈레톤을 렌더링하는 useEffect 영역]

```tsx
// plan id 바뀔 때마다 이전 스토어 상태 초기화
useEffect(() => {
  resetPlan();
}, [uuid, resetPlan]);

useAutoSavePlan(uuid);
usePlanSync(uuid);
```

[스켈레톤 UI]

트러블 슈팅

dnd-kit의 컨테이너 간(장소 리스트↔Day 카드) 및 Day 내 카드 순서 변경 오작동

상황

사용자가 후보 장소 리스트에 있는 카드를 Day별 일정 영역으로 드래그해 옮기거나,
같은 Day 안에서 카드의 순서를 변경하려 할 때 드롭 영역을 인식하지 못하거나 순서값이 엉켜 순서가 안 바뀌는 현상이 발생했습니다.

원인 파악

dnd-kit 라이브러리 특성상 후보 장소 리스트와 Day별 리스트라는 다중 컨테이너 환경에서 드롭 타깃이 컨테이너 자체의 ID인지,
내부 카드 ID인지를 명확히 식별하지 못해 이벤트가 작동하지 않았습니다.
특정 Day에서 카드를 빼거나 중간에 옮겼을 때, 기존 카드들의 순번이 연쇄적으로 재정렬되지 않고 중복/누락되어 순서 데이터가 무너짐을 알았습니다.

해결 방안

- **컨테이너 추적 함수(findContainerDay) 도입**
  - 타깃 id가 컨테이너 자체인지 내부 카드인지를 구분하고, 카드일 경우 속한 Day를 역추적하여 교차 이동인지 같은 Day 내 순서 변경인지를 명확히 분기 처리했습니다.
- **순서 자동 재정렬 및 Zustand 스토어 연동**
  - 카드가 다른 Day로 이동할 때 출발지 Day의 순서를 1부터 다시 매겨 메우고, 도착지 Day의 맨 뒤 순서를 부여했습니다.
  - 해당 Day 내 카드 전체의 순서를 일괄 재할당하여 일관성을 확보했습니다.

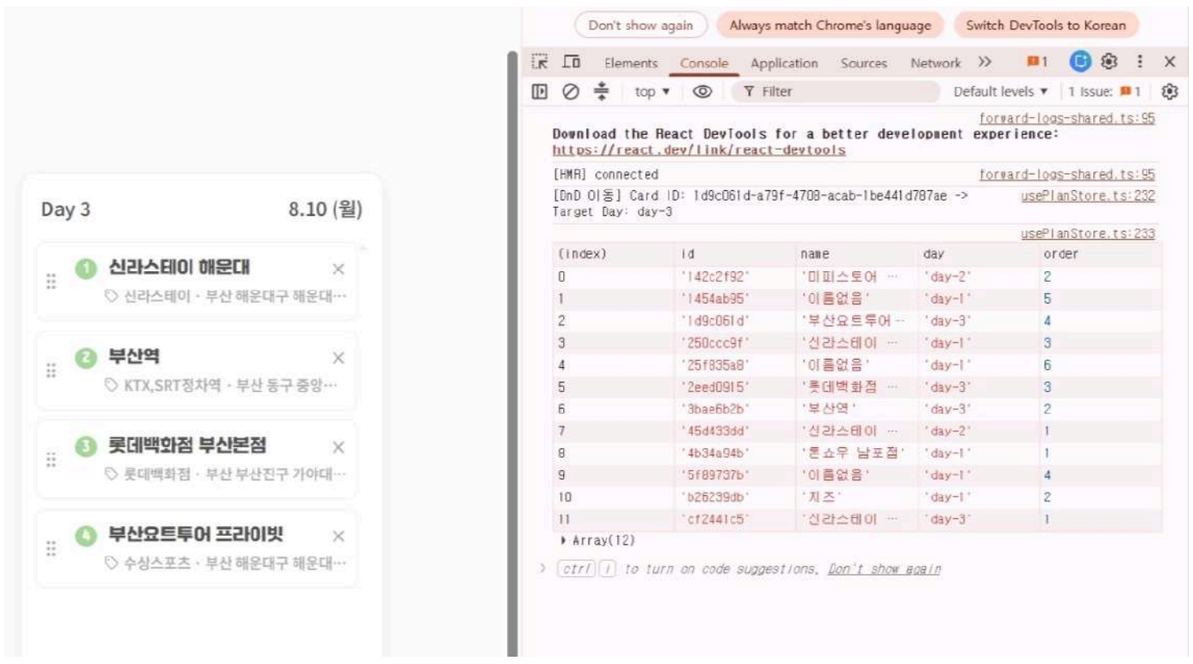
결과

드롭 실패와 순서 꼬임을 각각 다른 층에서 구조적으로 막았습니다.

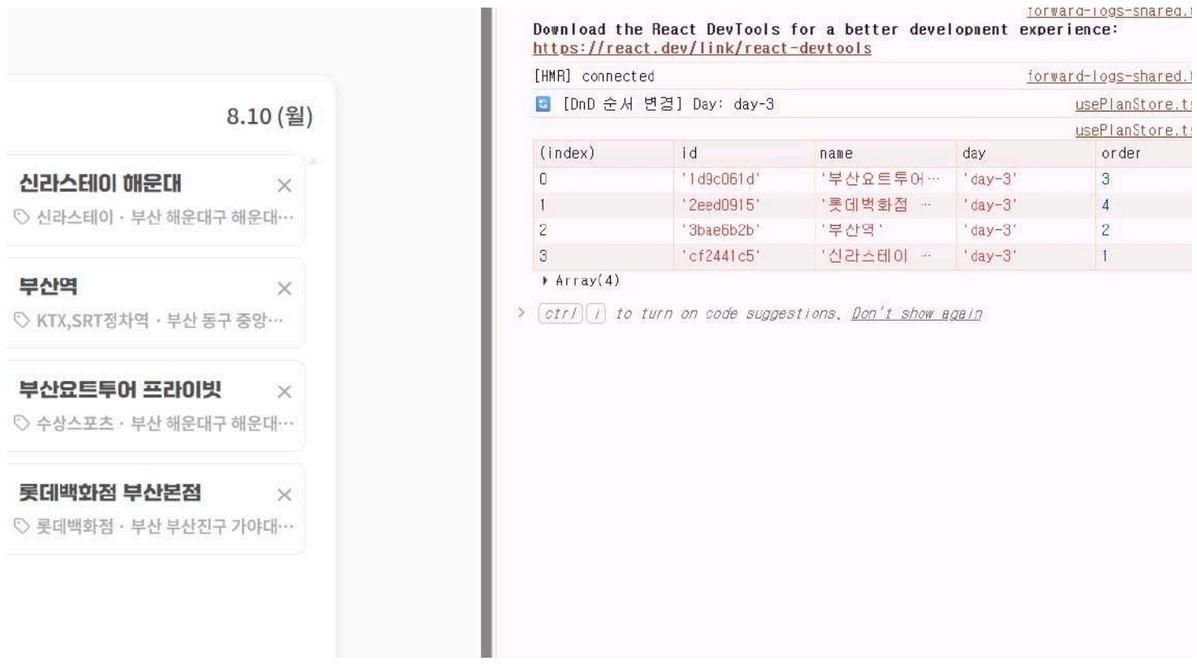
- 드롭 타깃 식별
  - findContainerDay로 타깃 id가 컨테이너 자체인지 내부 카드인지 판별하고, 카드면 속한 Day를 역추적합니다.
  - 이로써 "교차 이동"과 "같은 Day 내 순서 변경"이 명확히 분기됩니다.
- 순서 데이터 무결성
  - 이동 후 출발지 Day의 순서를 1부터 다시 매기고, 도착지 Day에는 맨 뒤 순서를 부여한 뒤 해당 Day 전체를 일괄 재할당합니다.
  - 순번을 증분으로 보정하지 않고 매번 전량 재계산하므로 중복이나 갭이 발생할 수 없습니다.

검증은 콘솔에서 이동 후 카드의 날짜·order 값이 의도대로 재계산되는지 직접 확인했습니다.

[드래그 앤 드롭 후 카드 데이터의 날짜와 순서가 재계산된 콘솔 스크린샷]



같은 Day3안에서 부산요트투어 프라이빗 카드를 3번째로 옮겼을 때 order값이 재계산됨.



회고

상태 관리 라이프사이클과 동기화 아키텍처를 사전에 엄격히 설계해야 했습니다

프로젝트 초반, 상태 관리 스토어의 범위와 서버/소켓 이벤트와의 동기화 우선순위를 명확히 규정하지 않은 채 구현을 시작했습니다.
그 결과, 페이지 라우팅 시 이전 데이터의 잔상이 순간적으로 노출되거나,
실시간 수신 이벤트가 사용자가 작성 중이던 임시 데이터를 덮어써 버리는 버그를 맞닥뜨렸습니다.
프론트엔드 중심의 실시간 시스템일수록, 화면 구현에 앞서 상태의 라이프사이클과 데이터 흐름을 얼마나 정교하게 설계했느냐가
개발 전체의 안정성을 좌우한다는 것을 깊이 깨달았습니다.

기능 구현 보다 중요한 것은 다중 컨테이너 및 비동기 환경에서의 엣지 케이스 방어였습니다

드래그 앤 드롭 기능이나 초대 링크 기반 유저 유입 시스템처럼 기능 동작 자체를 완성했다고 생각했을 때가 시작이었습니다.
막상 다양한 시나리오로 테스트해보니 다중 컨테이너 간 droppable 타깃 추적 오류,
순서 인덱스의 중복 및 갭 발생 등 버그가 계속해서 터져 나왔습니다.
헬퍼 함수를 통한 드롭 타깃 역추적, 자동 재정산 알고리즘 구축, Skeleton UI 도입을 통해 비동기 공백을 방어했습니다.
주요 로직의 정상적인 단일 흐름만 보고 작성을 마치는 것이 아니라, 복잡한 사용자 인터랙션과 예외 케이스를 얼마나 촘촘하게 제어했는지가
전체적인 서비스 품질을 결정한다는 것을 배웠습니다.

지난 회고를 바탕으로 이번 프로젝트에서 이렇게 바꿨습니다

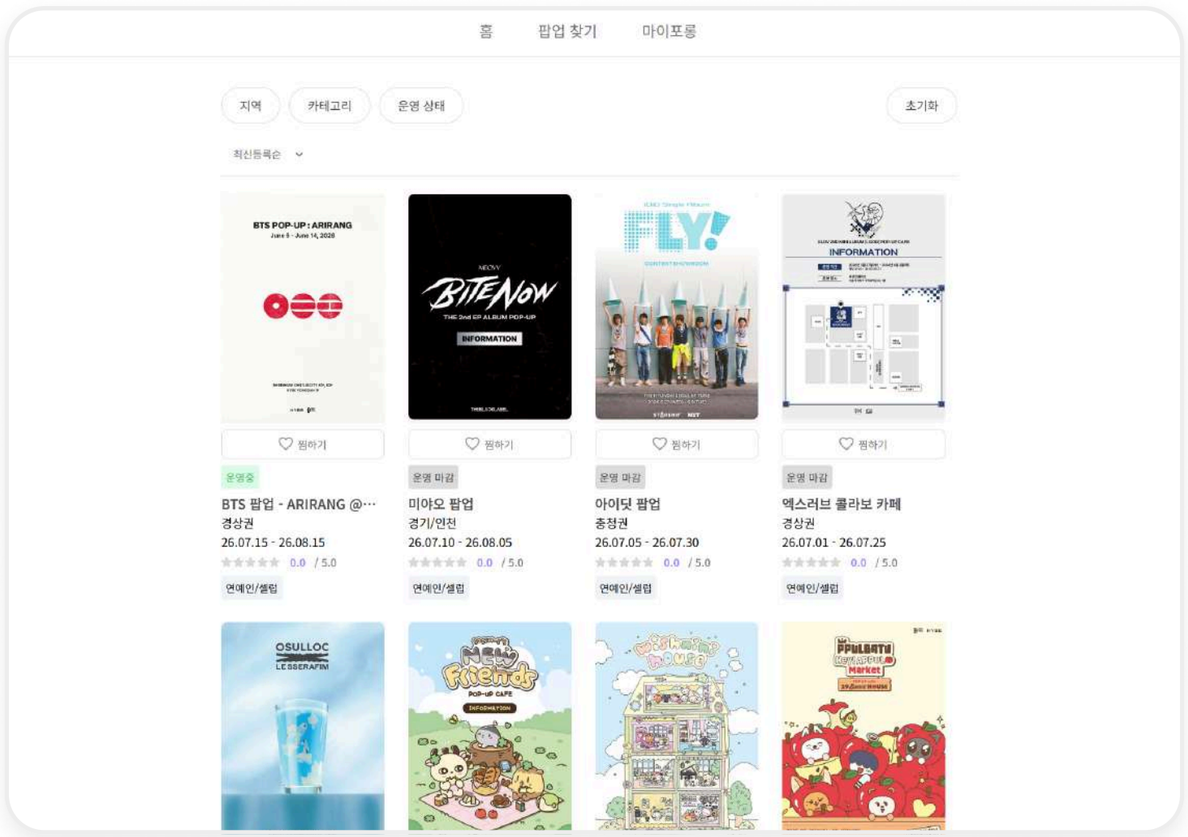
PORONG에서는 정상적인 사용 흐름만 보고 완성됐다고 판단했습니다. 그러다 실제 기기에서 통합 테스트를 하고 나서야 런타임 에러와 엣지 케이스를 뒤늦게 발견했습니다.
그래서 NolMong에서는 기능이 동작하는 시점을 완성으로 보지 않고, 여러 시나리오로 테스트하는 단계까지를 구현 범위에 넣었습니다.
드래그앤드롭과 초대 링크 유입 기능을 다양한 순서와 조합으로 테스트했고, 그 과정에서 다중 컨테이너 사이에서 드롭 타깃을 잘못 추적하는 오류
 순서 인덱스가 중복되거나 비는 문제를 찾아냈습니다.
이 경험으로 '기능은 실패 상태까지 설계해야 끝난다'는 기준이 생겼습니다.

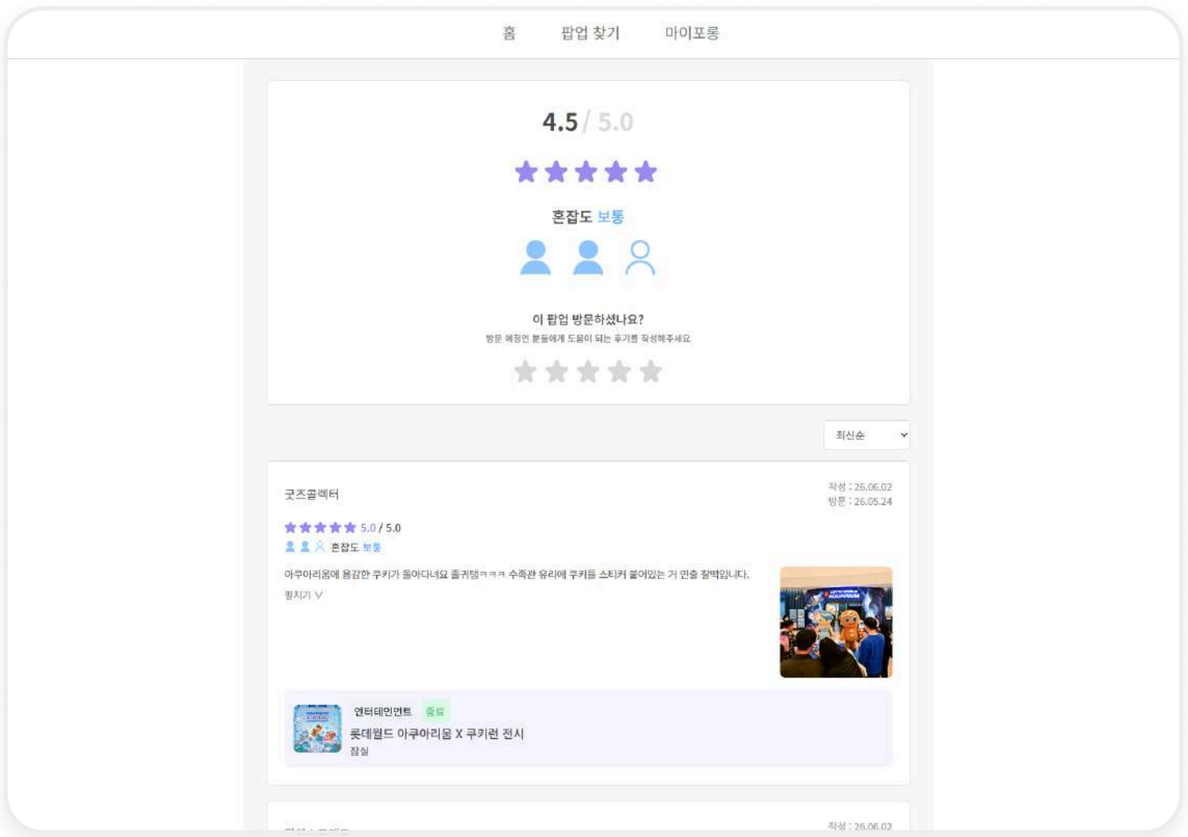
다음 프로젝트에서는 이렇게 바꿔볼 생각입니다

다음 프로젝트에서는 동시성 작업이나 교차 인터랙션이 발생하는 핵심 기능에 대해서는 개발 단계부터 에러 경계와 QA 체크리스트를 정량화해 둘 생각입니다.
이후 리팩토링 단계에서는 복잡한 분산 이벤트 구조를 간소화할 수 있는 대안 기술 스택까지 시야에 두고 비교 분석하여,
성능과 가용성을 모두 잡는 완성도 높은 아키텍처 파이프라인을 구축할 것입니다.

PORONG

Project - 3





서비스

팝업스토어 통합 정보를 제공하고,
방문 리뷰 작성, 재미 요소를 갖춘 팝업스토어 통합 정보 제공 웹사이트.

서비스 사이트 링크

(데이터가 로딩되는데 1분정도 걸릴 수 있습니다.)

작업 기간

2026. 05. 22 - 2026. 06. 04

문제 상황

팝업 스토어 정보가 개인 SNS나 개별 홈페이지에 분산되어 있어
사용자가 원하는 정보를 탐색하는 데 높은 피로감을 느낌.

방문 전 현장 상황(대기 줄, 혼잡도 등)을 파악하기 어려워
사용자가 불필요한 대기 시간을 낭비하게 됨.

목표

분산된 팝업 스토어 정보를 한눈에 볼 수 있는
통합 정보 및 검색 서비스를 제공.

유저들의 자발적인 방문 리뷰를 기반으로 현장 혼잡도와
실시간 정보를 공유하는 생태계를 조성하게 함.

리뷰 작성 시 리워드(키링 등)를 증정하는 보상 체계를 마련하여
유저의 지속적인 커뮤니티 활동을 유도.

‘HTML, CSS, JS, Spring boot, MySQL만을 활용하기’ 주제에 맞춰
탄탄한 웹 서비스를 완수하는 것을 목표로 함.

기술 스택

HTML CSS3 JavaScript
Spring boot Java MySQL
Kakao Map Figma Notion

구성원

FullStack 팀장 (팀원 3명)

기여도 40%

관련 활동

LG U+ 유레카 부트캠프 프론트엔드

산출물

Github Figma 협업 노션 API 명세서 소개 영상 발표 자료

담당 역할

Design

프로토타입 제작 (Figma)

UI/UX 디자인 (Figma)

FrontEnd

프론트엔드 아키텍처·협업 환경 리드

프론트엔드 개발 일정 관리 및 업무 배정

메인 / 찾기 / 검색 기능 및 페이지 구현

Kakao Map 연동

REST API 연동

예외 처리 및 UX 최적화

프로젝트 시연 및 발표

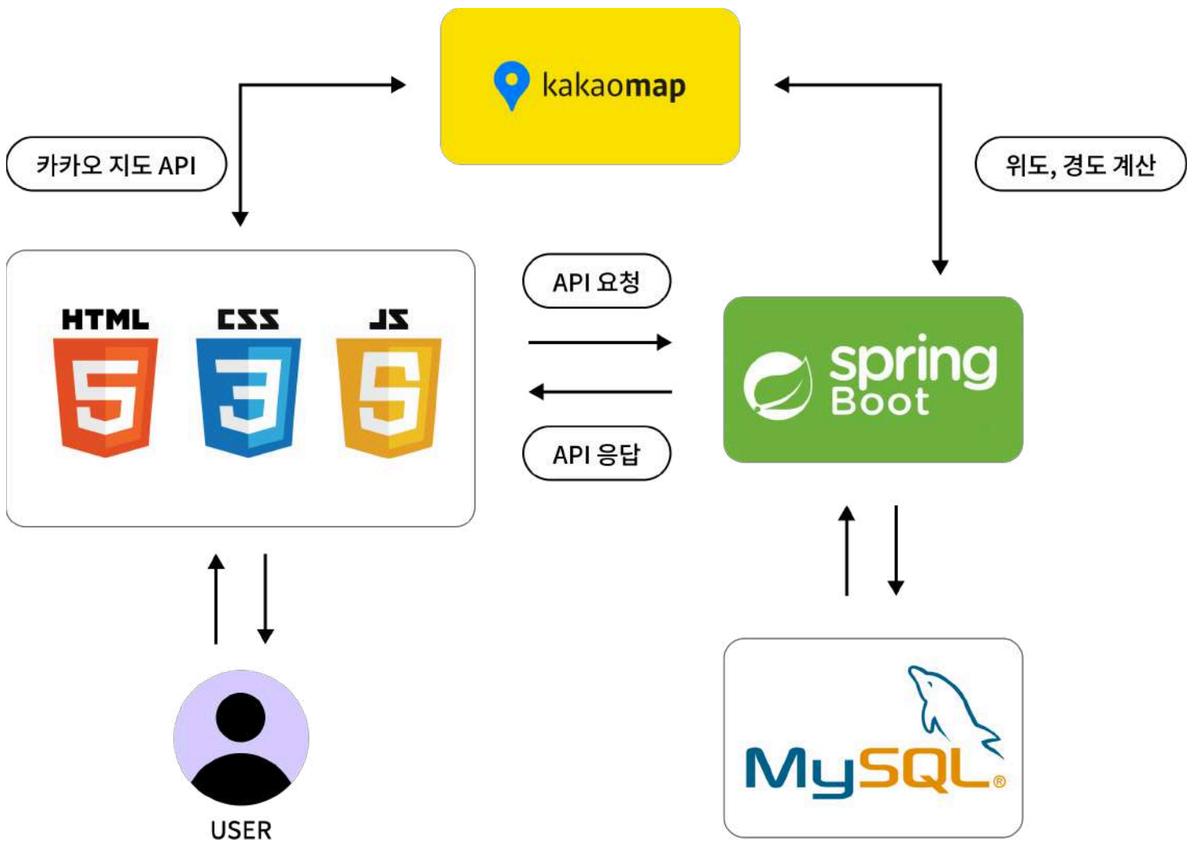
BackEnd

API 명세서 및 DB 정의서 작성

리뷰 CRUD 기능 개발

찜하기 등록 및 해제, 찜 목록 조회 기능 개발

시스템 구조도



핵심 구현

Vanilla HTML 환경에서 Base Template 설계

사용하게 된 맥락

React 등의 프레임워크와 달리, Vanilla HTML/CSS/JS 환경은 컴포넌트 단위의 분리 및 재사용이 어려워,
페이지가 추가될 때마다 Header, Footer 공통 UI 마크업 코드를 매번 복사·붙여넣기 해야 하는 한계가 있었습니다.
이는 단순 중복을 넘어, 추후 글로벌 스타일 변경 시 모든 HTML 파일을 일일이 수정해야 하는 에러를 유발하고
유지보수 비용을 증가시키는 요인이었습니다.
풀스택 팀원 3명이 각자 화면 개발에만 집중할 수 있도록 프론트엔드 초기 구조의 표준화가 필요하다고 판단했습니다.

수행

웹 표준 기술만을 활용해 초기 개발 효율을 높이고자 표준 구조를 설계했습니다.

- base-template.html 개발
  - Header와 Footer 고정 레이아웃을 미리 개발 후,
    본문 영역만 즉시 교체하여 개발할 수 있는 공통 템플릿 파일을 설계해 팀원들이 바로 복사해 사용할 수 있도록 배포했습니다.
- global.js 공통 인터랙션 모듈화
  - 페이지 이동, 가로 스크롤 등 전체 화면에서 공통으로 작동해야 하는 전역 UI의 핵심 인터랙션 로직을
    단 하나의 스크립트 파일로 통합 관리했습니다.

결과

공통 영역에 수정 사항이 발생하더라도 공통 인터랙션(global.js) 한 곳만 변경하면 전체 페이지에 반영되는 구조를 확립했습니다.
공통 인터랙션(global.js)과 가이드라인을 전역으로 구현하여,
화면 전환 시 발생할 수 있는 레이아웃 깨짐 현상을 방지하고 UI 일관성을 유지했습니다.

[base-template.html]

```html
<!doctype html>
<html lang="ko">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>PORONG</title>

    <!-- 모든 페이지 공통 스타일 (폰트, 여백 초기화) -->
    <link rel="stylesheet" href="/css/global.css" />

    <!-- 이 페이지 전용 CSS가 필요하다면 여기에 추가 (예: main.css 나 board.
    css) -->
    <!-- <link rel="stylesheet" href="/css/main.css"> -->
  </head>
  <body>
    <!-- [공통 헤더] 모든 파일에 동일하게 복붙해서 사용합니다. -->
    <header class="common-header">
      <div class="header-container">
        <div class="header-logo">
          <a href="/index.html" class="logo font-point">PORONG</a>
        </div>

        <input type="checkbox" id="menu-toggle" class="menu-checkbox" />

        <nav class="nav-container">
          <div class="center-menu">
            <a href="/index.html">홈</a>
            <a href="/pages/explore.html">팝업 찾기</a>

            <a href="/pages/mypage.html">마이포롱</a>
          </div>
        </nav>

        <div class="header-right-group">
          <a href="/pages/search.html" class="search-btn" aria-label="검색">
            <svg
```

[global.js]

```js
// 공통된 JS파일을 넣는 파일입니다.
// 로그인이나 따로 분리가 가능한 기능들은 따로 js파일을 만들어 주세요
window.API_BASE_URL = "http://localhost:8080/api";
const API = window.API_BASE_URL;

// 실시간으로 로컬스토리지에서 최신 유저 쿼리를 만들어주는 공통 함수
// 홈페이지, 찾기 페이지 등에서 팝업이 찜하기가 되어있는지 판단할 때 호출
function getUserQuery() {
    const loginUser = JSON.parse(localStorage.getItem("loginUser"));
    if (!loginUser) return "";

    const currentUserId = loginUser.userId || loginUser.id;
    // 백엔드가 컨트롤러에서 받는 이름(user_id)으로 매핑
    return currentUserId ? `&user_id=${currentUserId}` : "";
}

// 날짜 변환 함수
function formatDateString(isoString) {
    if (!isoString) return "";
    const datePart = isoString.split("T")[0];
    const [year, month, day] = datePart.split("-");
    return `${year.slice(2)}.${month}.${day}`;
}

// 헤더 로그인 상태 업데이트
function updateHeader() {
    // 로그인/회원가입 페이지에서는 실행 안 함
    if (
        window.location.pathname.includes("/pages/login") ||
        window.location.pathname.includes("/pages/register")
    )
        return;

    const user = JSON.parse(localStorage.getItem("loginUser"));
    const loginLink = document.querySelector(".login-link");
```

핵심 구현

디자인-코드 동기화를 위한 디자인 시스템 구축 (CSS 사용자 지정 변수 도입)

사용하게 된 맥락

프로젝트의 UI/UX 기획과 Figma 화면 설계를 전담하면서,
디자인 단계에서 정의한 톤앤매너와 컴포넌트 규칙이 실제 코드 구현부에서 분산될 수 있다는 위험을 인지했습니다.
3명의 개발자가 각자 스타일을 스타일시트에 하드코딩하게 되면, 미세한 컬러 오차나 타이포그래피 규칙 붕괴가 발생하여
전체 서비스의 시각적 완성도가 떨어지고 공통 디자인 수정 시 막대한 리소스가 소모될 우려가 있었습니다.

수행

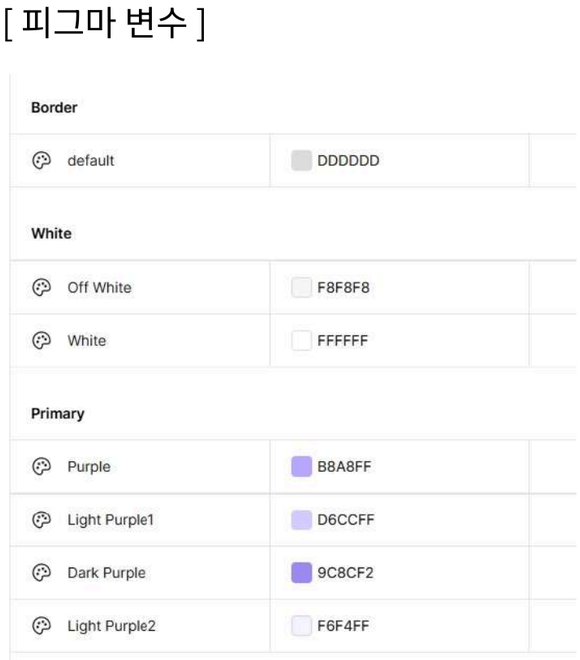
Figma 디자인 시스템을 소스코드와 동기화하기 위해, global.css 내에 CSS 사용자 지정 변수를 적극적으로 도입했습니다.

- :root 기반의 스타일 가이드 변수화
  - Figma에서 지정한 메인/서브 컬러, 테마별 배경색, 단계별 타이포그래피, 폰트 두께, 여백 등을
    :root 클래스 내에 변수로 선언했습니다.
- 스타일 컨벤션 통일
  - 팀원들이 스타일을 적용할 때 구체적인 색상 코드(ex. 3B82F6) 대신,
    사전에 정의된 사용자 지정 변수(ex. var(—primary-color)) 만을 사용하여 화면을 구성하도록 프론트엔드 스타일 규칙을 리드했습니다.

결과

스타일 변경 요청이나 다크모드 등 테마 확장성을 고려한 유지보수 효율성을 확보했습니다.
실제로 특정 브랜드 컬러를 변경할 때 :root 내의 변수 값 하나만 수정하면 프로젝트 전반의 모든 페이지에 실시간으로 반영되는 구조를 완성했습니다.
결과적으로 Figma에 정의한 컬러·타이포그래피·여백 토큰 17개를 :root 변수로 1:1 대응시켜, 팀원 3명이 각자 색상 코드를 적어 넣던 방식을 변수 참조로 통일했습니다.

[피그마 변수]



[global.js]

```
:root {
    /* 피그마 디자인 시스템 변수 등록 */
    --color-primary: #B8A8FF;
    /* 메인 보라색 */
    --color-primary-dark: #9C8CF2;
    /* 메인 보라색2 */
    --color-primary-light1: #D6CCFF;
    /* 메인 보라색-light */
    --color-primary-light2: #F6F4FF;
    /* 메인 보라색-light */

    --color-text-main: #4C4C4C;
    /* 기본 메뉴/본문 회색 */
    --color-text-secondary: #757575;
    /* 더 어두운 텍스트 ( 필요시 ) */

    --color-white: #ffffff;
    /* 흰색 배경 */
    --color-offwhite: #F8F8F8;
    /* 오프화이트 색상 */

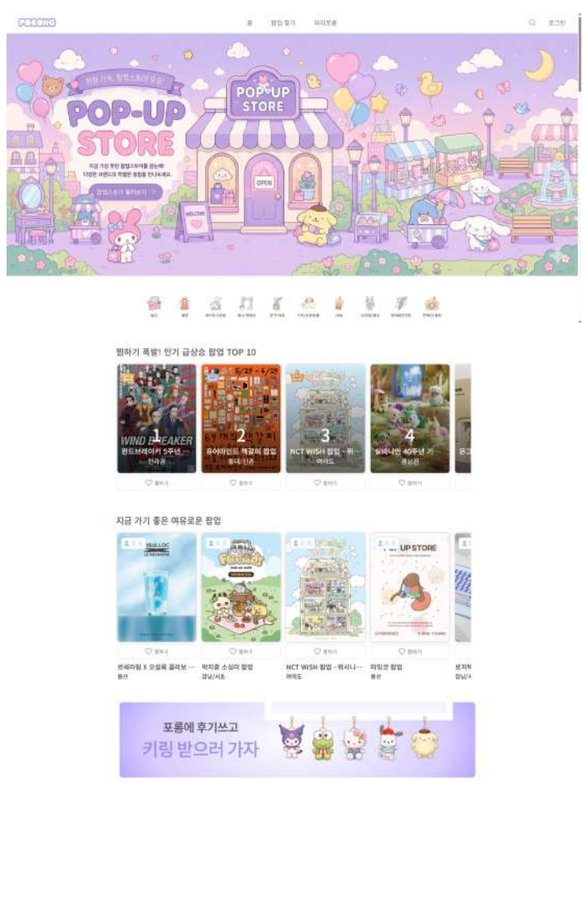
    --color-border-default: #DDDDDD;
    /* 기본 테두리 색상 */

    /* 카테고리 운영 태그 배경색 */
    --color-category-bg: #F1F5F9;
```

[피그마 프로토타입]



[실제 웹사이트 사진]



트러블 슈팅

Multipart/Form-Data 통신 규격 오류로 인한 후기 수정 기능 마비

상황

리뷰 수정 페이지에서 기존 텍스트와 변경된 사진 바이너리 파일을 서버로 전송할 때 415 (Unsupported Media Type) 에러와
405 (Method Not Allowed) 에러가 연쇄적으로 발생하며 수정 처리가 작동하지 않는 위기를 겪었습니다.

원인 파악

백엔드의 수정 요청 규격을 단순 JSON 방식으로 오인하여 데이터를 전달했으나, 서버에서 데이터 전송 방식을 거부해 에러가 발생했습니다.
서버 소스코드를 분석한 결과, 백엔드 API가 단일 JSON 방식(application/json)이 아닌
이미지 파일과 텍스트 데이터를 함께 받는 폼 데이터 방식(multipart/form-data)으로만 요청을 받도록 제한되어 있었습니다.

해결 방안

- FormData 동적 컨테이너 전환
  - 기존의 JSON 문자열 직렬화 방식을 없애고, 브라우저 내장 객체인 FormData 빌더를 도입하여
    텍스트 데이터(userId, content, rating, congestionLevel)를 개별 append 구조로 패킹했습니다.
- Content-Type 헤더 수동 지정 제거
  - fetch 요청 시 헤더에서 Content-Type을 의도적으로 지워버려,
    브라우저가 멀티파트 고유 경계선 규칙을 포함한 정석 폼 데이터 포맷을 알아서 헤더에 자동 인코딩 매핑하도록 유도했습니다.

결과

백엔드가 요구하는 멀티파트 바이너리 규격을 정확히 충족하여 415 및 405 차단 에러를 해결했습니다.
컴퓨터 내부의 실제 물리 파일 바인딩과 데이터 전송을 온전히 확보하여 후기 편집 기능을 정상 작동하도록 했습니다.

[수정 전 코드]

```
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify(requestBody)
    });

    if (!response.ok) throw new Error("리뷰 수정 통신 실패");
```

[수정 후 코드]

```
    // FormData 패킹
    const formData = new FormData();
    formData.append("userId", Number(userId));
    formData.append("content", contentText);
    formData.append("rating", Number(selectedRating));
    formData.append("congestionLevel", selectedCongestion);

    // 새 사진 파일을 컴퓨터에서 선택했다면 파일 바이너리를 탑재하고,
    // 선택하지 않았다면 보내지 않아 백엔드 image = null (required = false) 기본 사양 충족
    if (rawImageFile) {
        formData.append("image", rawImageFile);
```

회고

기능만 돌아간다고 끝난 게 아니었습니다

기능 구현을 마치고 끝났다고 생각했는데, 실제 기기에서 통합 테스트를 돌려보니 생각도 못 한 런타임 에러와 엣지 케이스들이 계속 튀어나왔습니다.
정상적인 사용 루트만 보고 완성됐다고 착각했던 것입니다.
예외 상황을 얼마나 촘촘하게 방어해뒀는지, 그리고 여러 각도에서 얼마나 테스트했는지가 결국 제품 안정성을 결정한다는 것을 뒤늦게 깨달았습니다.

다음 프로젝트에서는 이렇게 바꿔볼 생각입니다

이후, 기능 구현이 끝났다고 바로 손 떼지 않고, 실제 배포 환경과 최대한 비슷한 조건에서
시연 테스트를 여러 번 돌려보는 것을 필수 단계로 넣으려 합니다.
시연하면서 발견되는 UI/UX 병목, 통신 지연 같은 예외, 실사용자 입장에서 불편했던 지점들을 그때그때 흘려보내지 않고
QA 체크리스트로 정리해서 남길 생각입니다.
이를 기반으로 디버깅과 리팩토링을 순서대로 처리하면,
예외 상황에서도 잘 버티는 완성도 있는 결과물을 낼 수 있는 파이프라인을 만들 수 있을 것이라 생각합니다.

감사합니다.

이주현 | 웹 개발자

juhyunlee808@gmail.com

https://github.com/hana03030
Github

https://hana030.tistory.com/
블로그